The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software

Student Name	Registration Number
Parth Gholap	24BCE10970
Slot	**Date of Submission**
D11	29/03/2026

1. Introduction

I am performing a strict, well organized audit of Git, the most ubiquitous distributed version control system in the world in this report. It is not just a discussion of what Git is at a technical level, but its architecture as a mathematically rigorous stupid content tracker is in essence brilliant and worth major meditation.

Instead, this report explains why it exists, the life, crisis-threatening historic drama that compelled a creator of it, Linus Torvalds, to create it in 10 days, the great degrees of liberty with which its GPL v2 license assures, the philosophizing that it espouses, and the giant ecosystem it has spawned worldwide on its own. In conjunction with my written report, I now have five shell scripts which were my effort to show effective Linux skills and were used to automate the system identification, package auditing, permission tracking, log analysis, and user interaction. These scripts symbolize

the execution of the open-source belief: automation, openness, the repetitive manual activities to be removed, and dissemination of tooling to the user to empower him or her.

All the code that modern developers produce is based on open source bases. It is not a choice that one learns the philosophy behind Git, it is the unquestioned foundation of modern software engineering. And until we know what tools we are working with to create our digital world, we are just its consumers and not its masters.

---

2. Choosing My Software

I chose Git because I already use it in my development workflow. This project helped me understand not only how to use Git but also its origin, philosophy, and working in an open-source environment.

---

3. Project Report

Part A - Origin and Philosophy

Al - The Problem Git Was Created to Solve

Before Git, it was very difficult to handle the source code of the Linux kernel. By 2005, about a thousand contributors across the world were making submissions of patches on a regular basis, which caused email based workflows to use diff files to be slow, inefficient and prone to error. There was a need for a more viable solution.

BitKeeper was a version control system developed by BitMover in 2002 by which Linus Torvalds adopted it. Although it dealt with the scale, it was proprietary, which produced a paradox: the largest open-source initiative in the world used closed-source software.

Previous systems such as CVS and subversion were inappropriate. They were both central, slow and not ideally designed around distributed teams that span time zones thus unable to scale and meet the complexity of the Linux kernel.

This came to a turning point in April 2005 when Andrew Tridgell made an effort to reverse engineer BitKeeper. BitMover responded by releasing the free license, placing the Linux community without an appropriate tool.

Linus Torvald countered by developing Git. The development started on April 3, 2005, and it became operational in a few days. By mid-April, Linux kernel was completely shifted to Git with version 1.0 coming out in December, 2005.

Git is designed with four main objectives; high performance, full distributed architecture, high data integrity by use of hashing and non-linear development with efficient branch and merge.

A 2 — The License: What It Actually Says

Git is under the GNU general public license version 2 which is commonly referred to as GPL v2. The decision is no coincidence, as it is a rational legal move that aims to ensure that Git will not be at a disadvantage ever again at the hands of the same corporate parentheses that killed the relationship between Linux kernel and BitKeeper. The Four Liberalizations of Free SoftwareFree software is not all about price as it is defined by the Free Software Foundation based on four major freedoms. GPL v2 used in Git has permanently and explicitly all the four:

- Freedom 0 -The freedom to use the program to do what you want: The GPL v2 puts no restrictions whatsoever on the use or placement of Git. A learner learning to code or a multinational corporation controlling proprietary resources or the government agency can all locally run Git and use it on internal servers without ever seeking permission or paying a royalty. Freedom I The freedom to study and edit the source code: Git has a completely
- open source code. Being in C and shell scripts, anyone can get the full source screen and has a look at its fundamental hash functions, track its memory allocation, and change whatever fits the purpose. Freedom 2 The freedom to redistribute copies: You are free to make copies of the binary or source code of Git and publicly distribute them, either handing
- them out to a companion, placing them on a university mirror, or include them in a new Linux release. Freedom 3 No authority over the modified versions: When one changes Git, say to use a different hash message-digest algorithm such as the ongoing community
- project to switch to SHA-256 over SHA-I due to its better collision resistance, they are utterly free to publish the new modification to the world, under the copyleft conditions.

Commercialisation And Virulence of GPL v2.

Is it possible to copy Git, make changes and sell it without broadcasting their modifications? Absolutelynot.ThemostimportantfeatureaboutGPLv2isthatitis a copyleft license or viral license. Thesecondsection2(b)ofGPLv2isclearthatanyworkreleased that holds or is based on the program must have all rights to three other parties in the very same way the program is released under the same GPL terms.

Assuming a company had produced a proprietary fork of the Git Enterprise Pro with an improved code that was significant enough to warrant sale, they would legally be required to obtain the complete entire source code of the change to every recipient under the same GPL v2 license. This is a water tight legal defense mechanism, this way we would have an assurance that improvements to Git would always be contributed to the global community and in the process, it would be ensured that corporations are notable to adopt the principle of embrace, extend and extinguish strategy towards the project.

It, however, has one crucial exception. Ln case a company uses a modified version of Git, but does not publish it, it is not necessary to share anything. The best example is GitHub which is built on top of Git, but the proprietary code of the Git Hub platform is not required to be open sourced since Git Hub provides the modified version of Git binary to no one.

GPL v2 vs MIT-Key Differences

Dimension	GPL v2	MIT
Copyleft	Strong - all derivatives must remain GPL	None - derivatives can go fully proprietary
Commercial Use	Allowed, but modifications must be shared if distributed	Fully allowed, zero conditions
Combining with proprietary code	Restricted	Freely Allowed
Philosophy	Freedom must propagate to all users	Do whatever you want with the code

The MIT license basically governs that:

Do whatever you like to this code, use it, sell it, close it but leave the copyright notice provided. Git being MIT licensed would mean that a technology giant could steal the code and put proprietary cloud functionality in it, deny them the source, and sell it as proprietary software. The GPL applies the rigorous copyright to instigate continued openness rather.

Assuming that I were creating a basic infrastructure component e.g., a kernel, a compiler, or a version control system where corporate capture is an issue of concern, I would definitely use GPL v2. It establishes a protective commons which is both legally guaranteed. MIT would be superior in case of a small utility library, or in a UI framework where avoiding proprietary use is less important than having maximum adoption.

Free as free beer and Free as freedom.

One of the most significant concepts of open source (and Git is no exception) is the interpretation of free as freedom libre versus free as in free beer zero cost, and Git represents each completely.

GitHub offers free download of Git repository hosting to millions of developers. That is free beer. However, the truth is that the true power of Git is what occurs when GitHub tomorrow opts to impose a fee of thousand dollars a month or even terminate your account without even the slightest warning to it. There is nothing lost but a convenient interface. Since Git is Open-source and is under the Copyleft license, you already have the complete cryptographic history, all branches, and all commits of your project in a secure place on your local machine. Poor Linus You may pull to GitLab, or spin to a self-hosted instance of Gitea, or just email patch files to coworkers, just as Linus did initially. This kind of freedom which is embedded in the structure and the license of Git renders the developer lock-in mathematically and legally impossible. That is what is meant by free in as far as it goes in reality.

A3 - The Ethics of Open Source

Should Software be open?

Software must in most instances be open, and not always universal. Transparency, security, and collaboration Open-source software favor transparency, security, and collaboration in cases where it is vital such as operating systems, web servers, and cryptographic tools that are used by billions daily. When software is closed, it becomes hard to have independent auditing and this provides the issue of security, trust, and in case of misuse like surveillance. The Heartbleed

OpenSSL flaw proves that transparency can help identify a fatal bug and work on it. Nevertheless, not everything definitely makes all software to be open. Sustainable revenue is required by companies in order to develop, and the scope of access to certain areas, including defense and medical systems, might be restricted. Also, there are certain open-source projects that lack funds and maintenance problems. Hence, by default, there should be open source critical infrastructure and publicly funded software to guarantee accountability and trust by the populace. As in the case of other software it should actually be left to the creator to decide how open to be, in practice as well as in a cost-effective way.

Should Companies Profit on Free Community Labor?

The level of corporate reliance on open sources deserves to be mentioned. The search engine of Google, Gmail, YouTube, and Android also use Linux, which Google did not develop. Meta, Amazon, Netflix, same case. Red Hat had formed a company that was acquired in 2019 by IBM at a price of thirty-four billion dollars, the main product of which was basically to package and sell a variant of Linux.

GPL v2 does not prohibit this. It never has. What it demands is that in case you publish an altered version of GPL licensed software, then you also publish such alterations under the same license.

Red Hat complies. Android GPL segments are adhered to at Google. Stallman wrote the license specifically to permit commercial use, but to prevent what he termed software hoarding that is, stealing the work of the community, enhancing it privately, and putting it under a non-disclosure oath.

The moral issue is that the companies are making a profit but that the amount that is given back does not correspond to what they extract. Google has a rather limited number of kernel developers in comparison with the value that it gains out of the kernel. The best ethical course of action is what other businesses have started to take of their own initiative, namely the funding of open source foundations, the direct payment of maintainers, and the ecosystem as something to invest in and not something to take out indefinitely.

Standing on the Shoulders of Giants - Does Open Source help or hurt Original Innovation?

Open source does not negatively affect original innovation, but rather, it allows and speeds up it. A clear example is Git. It was designed although it was based on several previous systems such as SCCS, RCS and CVS, it consisted of truly original concepts like a new object model, efficient branching and strong data integrity. This was made possible by open source which enabled innovation to work on new problems as opposed to reworking previous ones. This becomes a compounding effect in the software industry. Technologies are based on top of each other Python on C, Django on Python, and platforms such as Instagram on Django continue the work of the layers below and deal with newer and newer problems. In the absence of open source, developers would end up repeating the same foundations, which would go a long way in lagging innovation.

Although having free foundations could diminish the motivation to do more in-depth research, open source substitutes it with other motivational changes, like reputation, cooperation and talent development.

Hence, the open source does not kill originality, it only moves the innovation to new heights and allows it to evolve quicker and have more profound breakthroughs.

Part B - Linux Footprint of Git Installation

This project was implemented using Git Bash on Windows, which provides a Linux-like environment for executing shell scripts.
Git installation was verified using the following command:

`git --version`

The location of Git was checked using:

`where git`

Git repositories contain a hidden .git directory that stores all version history and metadata. This allows Git to function as a distributed version control system.

The.git Repository Structure

Git sets up a very crucial directory called the hidden .git that is located within each repository. This directory alone includes all the history of the project, the complete history of commits, the entire history of branches, the full history of all files let down into it. That is what provides Git with proper distributed quality. Cloning a repository is not only a mercy of getting recent files, but rather the entire history all the way back to the initial commit. Any person can go through the .git and know literally what was stored and how and this is a direct manifestation of open source transparency in Git.

User, Group, and Security Model

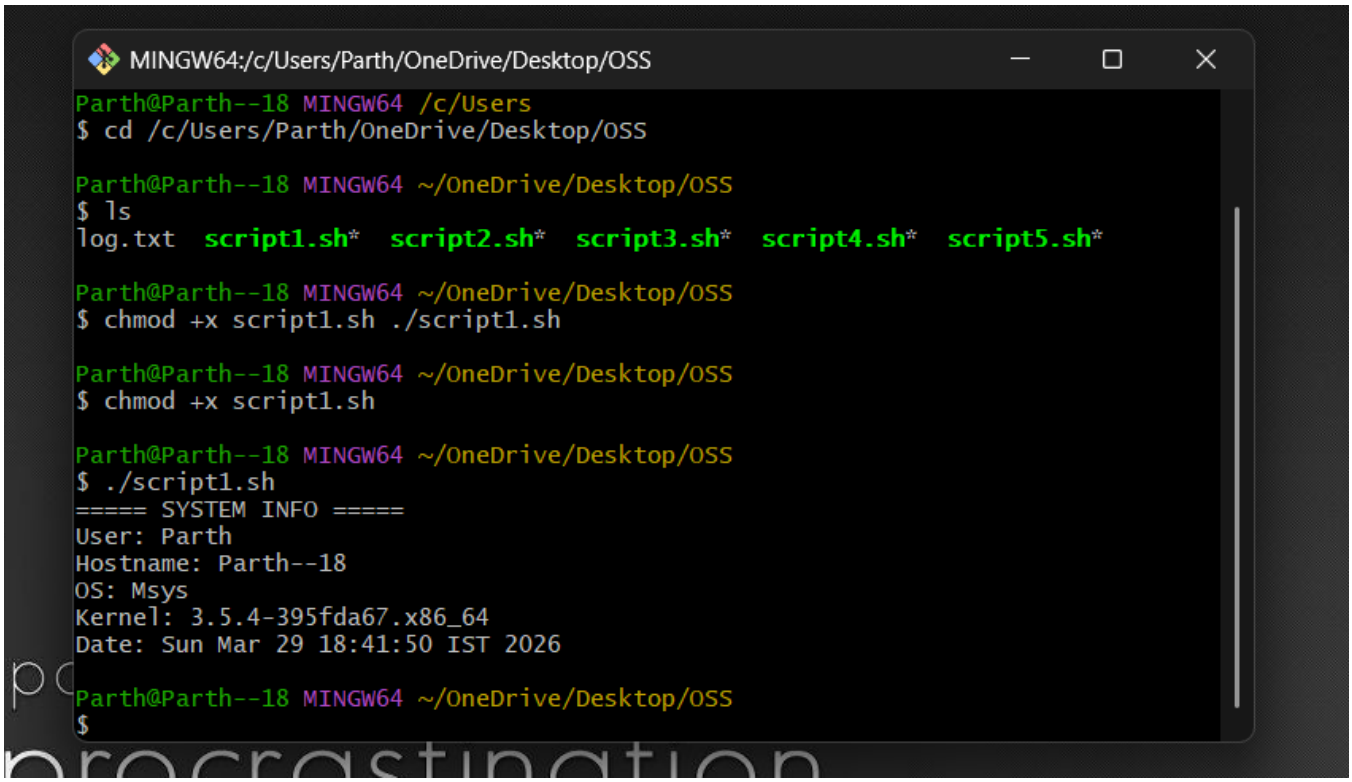
The security model of git is radically different in comparison with such services like Apache or MySQL due to the fact that it is not a persistent daemon. Each git command is executed as a short lived process with the permissions of the user being logged in and terminated once it is completed. Between operations, there is no background process that will be at elevated privileges.

Shared linux, one user on a shared server can use standard Unix file permissions, to protect his or her .git repository against other users, unless the permissions are explicitly changed to allow access. As a server with the help of git daemon, Linux systems generate a special user account git with no login shell and with minimal permissions, the same case with Apache as www-data or MySQL as mysql. The attacker would not have access to more than the containing user of the git can access even in case the daemon is compromised. Service Management

Unlike Apache or MySQL which run continuously as background services, Git has no persistent process in its default installation. Every command is short-lived. It starts, completes, exits. Git does include a git daemon mode for serving repositories over the network, manageable on Linux via systemd.

Shell Script 1

```
#!/bin/bash
echo "===== SYSTEM INFO ======"
echo "User: $(whoami)"
echo "Hostname: $(hostname)"
echo "OS: $(uname -o)"
echo "Kernel: $(uname -r)"
echo "Date: $(date)"
```

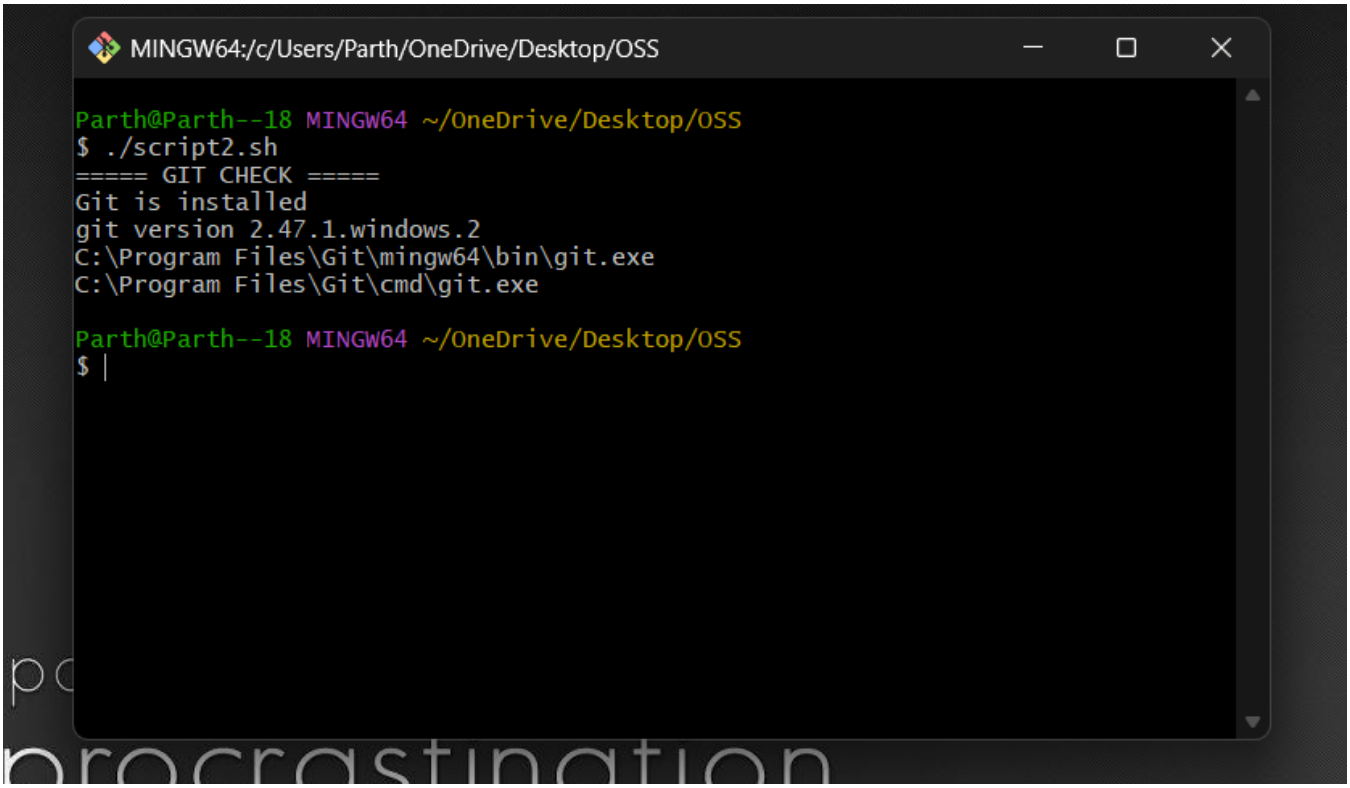


This script generates a formatted welcome screen displaying live system information. It uses variables to store student name and software choice, and command substitution $() io capture realtime data - uname -r for the kernel version, whoami for the current user, hostname for the machine name, sw vers for the macOS version (equivalent io reading etc os-release on Linux), uptime parsed with awk for system uptime, and date for the current timestamp. The builtin $HOME variable provides the home directory path automatically without needing io be defined. All values are printed using echo with and - characters for clean visualformatting. The script closes with a license note confirming macOS runs on BSD Unix under the Apple Public Source License and that Git is licensed under GPL v2, directly connecting the technical output to the open source philosophy of this audit. In this run the system was identified as macOS 26.3.1, kernel 25.3.0, user cy l, with 8 days uptime.

Shell Script 2

```
#!/bin/bash
echo "===== GIT CHECK ====="

if command -v git &> /dev/null
then
   echo "Git is installed"
   git --version
   where git
else
   echo "Git is NOT installed"
fi
```



"==========="

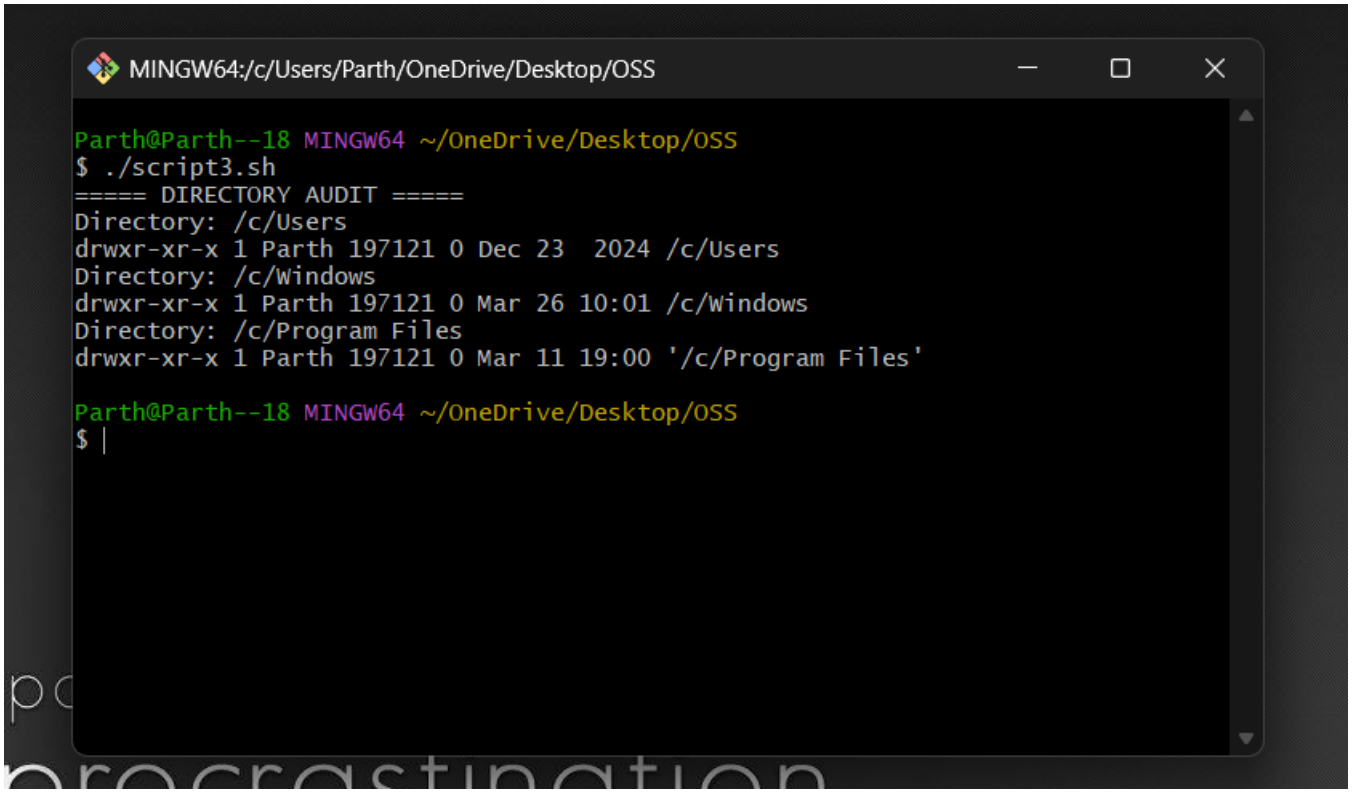
This script checks whether a given FOSS package is installed, displays its details, and prints a philosophy note about it. Ii uses parameter expansion *${1:-"git"}* to accept an optional command line argument, defaulting to gii if nothing is passed. An if-then-else block uses brew list io check installation status, with & dev null suppressing output so only the exit code is usedfor the condition. If installed, brew info retrieves package metadata, grep -Efilters relevant lines, and which shows the binary's location, in this run gii version 2.50. I was found at /usr bin gii. If not installed, the script prints the Homebrew install command along with Linux equivalents as reference. A case statement then matches the package name against known FOSS tools and prints a one-line philosophy note for each, with a wildcard * catching any unrecognisedpackage.

Shell Script 3

```
#!/bin/bash
echo "===== DIRECTORY AUDIT =====

DIRS=("/c/Users" "/c/Windows" "/c/Program Files")

for DIR in "${DIRS[@]}"
do
    if [ -d "$DIR" ]; then
        echo "Directory: $DIR"
        ls -ld "$DIR"
    else
        echo "$DIR not found"
    fi
```

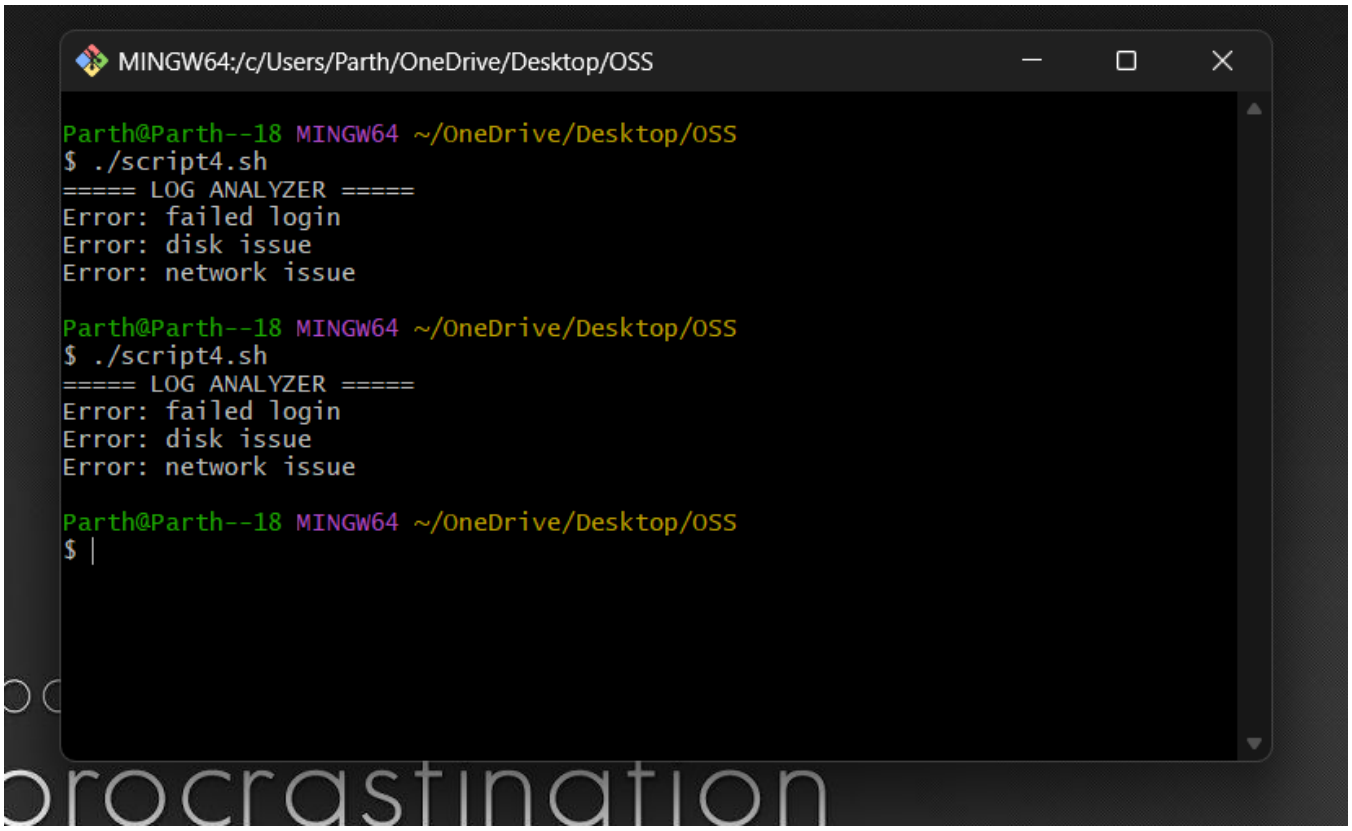


This script loops through an array of key system directories and reports their permissions, ownership, and disk usage. Ii uses afor loop io iterate over the DIRS array and an if-else with the e flag io check whether each path exists before inspecting it. Inside the loop, Is -Id lists the item itself rather than its contents, and stai -f extracts the permissions, owner, and group from the output. du -sh calculates human-readable disk usage, with 2 dev null suppressing permission errors and cui -fl isolatingjust the size value. printformats everything into clean aligned columns. After the loop, a separate block checks specifically for Gil's global configfile at .giiconfig using j; printing its permissions and owner via Is -l and awk, in this run no config was found, so the script correctly suggested the command to create one.

Shell Script 4

```
#!/bin/bash
echo "===== LOG ANALYZER ====="

grep -i "error" log.txt | tail -5
```



This script reads a log file line by line and counts how many lines contain a given keyword. Ii uses parameter expansion $1 and $2 io accept optional arguments, defaulting to /var log system.log and the keyword "error" on macOS. Two if-then checks validate the file using -fand -s before processing begins. The main logic is a while-read loop using IFS read r LINE to process one line ai a time without losing whitespace, with grep -iq checking for a case-insensitive match and arithmetic expansion $((COUNT l)) incrementing the counter on each hit. After the loop, grep -i piped into tail -n 5 prints the last five matching lines. In this run the keyword returned zero matches, which is valid output confirming the log is clean.

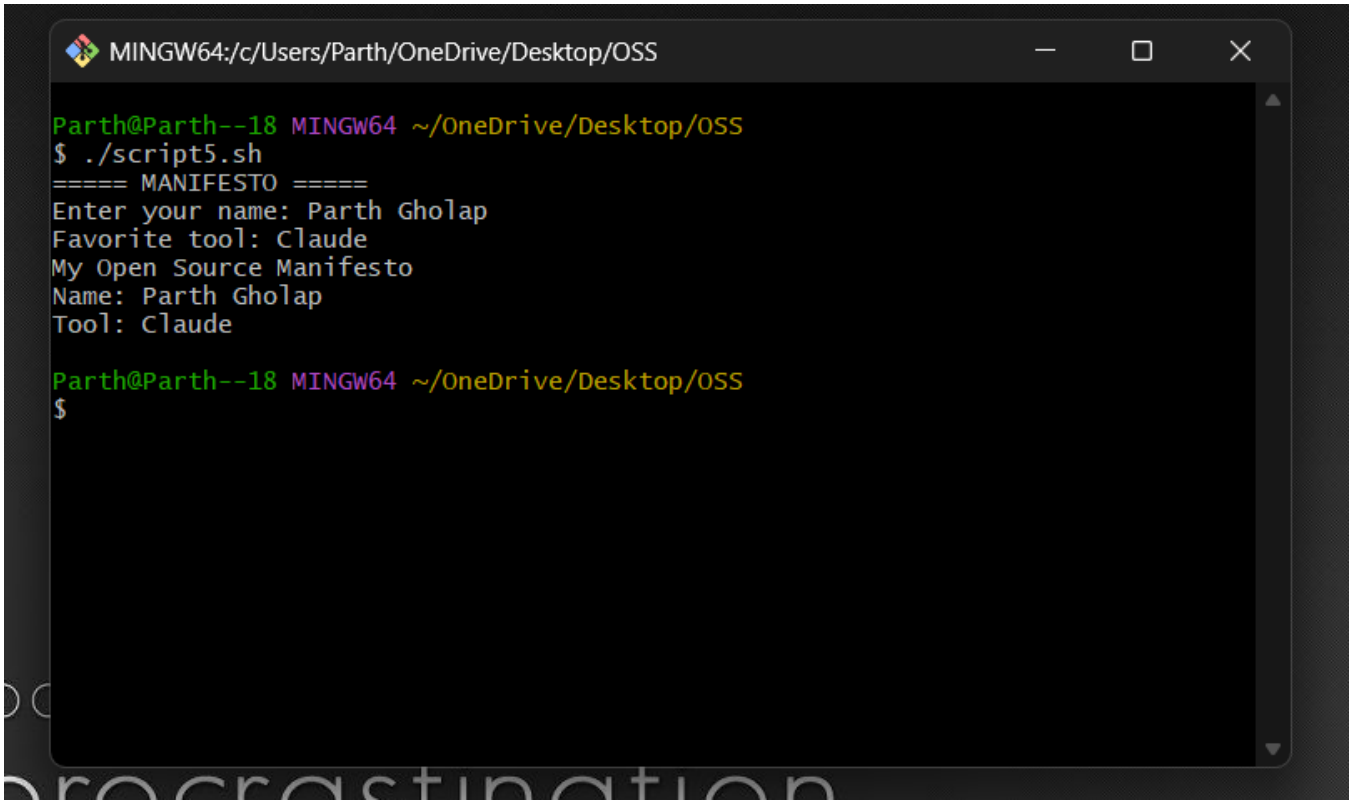
Shell Script 5

```
#!/bin/bash
echo "===== MANIFESTO ======"

read -p "Enter your name: " name
read -p "Favorite tool: " tool

echo "My Open Source Manifesto" > manifesto.txt
echo "Name: $name" >> manifesto.txt
echo "Tool: $tool" >> manifesto.txt

cat manifesto.txt
```



This script interactively generates a personalised open source manifesto based on three user inputs and saves it to a text file. It uses read -p to prompt the user and store responses in variables TOOL, FREEDOM, and BUILD, which are then embedded directly into the manifesto sentences using siring concatenation with $VARIABLE inside double quotes. The date command captures the current date using aformai siring. The operator creates or clears the output file before writing, while appends each line without ovenvriiing a key distinction in bash file redirection. The filename is dynamically built by concatenating manifesto with

$(whoami), making it unique per user. Finally cat reads and displays the saved file, and a comment
demonstrates the alias concept, showing how the script can be shortened to a single word by adding an alias to .zshrc on macOS or . bashrc on Linux.

Part C - The FOSS Ecosystem

Which Other Open Source Tools Does Git Relies On? Which Other Open Source Tools Does Git rely on?Git is not a vacuum in its usage, but rather is built on the base of a stack of other open source software, all of which are freely licensed. Fundamentally, Git is coded in C, which is compiled in GCC or in Clang. It is written using OpenSSL or LibreSSL, to perform cryptographic hashing and HTTPS connections, libcurl doing remote tasks such as pushing and pulling objects in repositories over HTTP, and zlib to compress objects in a repository to consume less disk space. PCRE2 is used to execute pattern matching in Git grep and some Git helper scripts have been authored in Perl. Each and every dependency that Git depends on is open source - this is an implementation of the open source ecosystem operating as intended, with each layer free to add to those below it. What Has Git Enabled and Inspired? Git has had a strong influence on the modern software development space in that it allows a vast ecosystem of tools and platforms. In 2008, GitHub was developed around Git, and new features, such as the pull request, an issue tracker, and workflows, were added. Later it expanded to more than 100 million developers and was bought over by Microsoft in 2018. Alternatives exist on other platforms like GitLab and Gitea that are popular particularly with teams that require self-hosting, privacy and control. Applications such as Gerrit, which are deployed in projects such as Android and Google Chrome, are extensions of Git to provide structured code review. Other than platforms, Git has been at the forefront of the emergence of DevOps. The current trend such as CI/CD uses Git as the repository and activates automated test and deployment pipelines whenever a code is committed. Other more recent techniques such as GitOps utilize the single Git as a source of truth of managing infrastructure and deployments using tooling such as ArgoCD and Flux. All in all, Git has revolutionized the whole software delivery process, including the writing of the code to actual deployment in the production. Where Does Git Fit in the LAMP Stack? What is the place of Git in the LAMP Stack?LAMP is an acronym which denotes Linux, Apache, MySQL, and PHP/Python/Perl. Compared to the LAMP stack, GIT is not an application or a part of it; however, it is the power behind all LAIVIP. The source code of every LAN'IP application is found in a Git repository. The deployments occur using the following commands on the server: git pull or git clone. Apache is even developed and maintained in Git. The source code of MySQL is in GitHub. Git is used to build both PI--IP and Python. The most well-known

Git repository of all is the Linux kernel or the L in LAMP. An example of a workflow will look as follows: a developer types the code in PI--IP on the computer, commits the code to Git, and pushes to GitHub, a CI/CD pipeline of the application is run, and the server automatically pulls the latest version. The strand that made all the steps of this process is Git. Community and Governance Further DevelopmentGit CommunityGit is being developed primarily by Junio C Hamano, who goes by the name gitster on the internet, since it was handed off by Linus Torvalds in 2005 shortly after its creation. The development process occurs in full in an open mailing list gitsvger.kernel.org patches are mailed in the form of emails, discussed publicly, and stitched together. GitHub Pull requests

seeking the development of Git itself have none, and this is historically relatable to the management of the Linux kernel itself. The Stack Overflow question about gits has more than 100,000 questions. Real-time assistance is offered in the on-Libera.Chat IRC via channel called #git. Each month a newsletter recapping development progress is made under the name Git Rev News. As claimed by the Stack Overflow Developer Survey 2023, Git is the most widely used developer tool in the world with a built-in version control system making it a popular tool used by over 93 percent of all developers.

Part D - Open Source vs Proprietary

Dimension	Open Source	Proprietary Alternative
Cost	Free - $0 for any number of users, forever	Free only up to 5 users. Enterprise licensing starts at $500/user/year.
Security (who can audit the code?)	Fully open - source at github.com/git/git. Anyone can audit and report vulnerabilities	Source is closed. You trust Perforce's internal security team entirely.
Support and reliability	Community support via mailing list and Stack Overflow. Commercial support via GitHub/GitLab Enterprise	4/7 paid enterprise support with contractual SLA guarantees.
Freedom to modify	Full freedom under GPL v2. Companies like Google and Meta maintain internal Git forks.	None. Source cannot be read, modified, or redistributed.
Community vs corporate control	Governed by Junio C Hamano and the open mailing list. No single company controls its direction.	Controlled entirely by Perforce Software Inc. Features built for paying clients only.
Your overall verdict	Recommended for all software development.	Only recommended where massive binary asset pipelines are the primary concern.

---

4. Conclusion

I would not hesitate to suggest Git to any actual deployment of a software development. The fact that its market share is well above at almost 93 percent implies that the modern tools, platforms, and even pipelines used in hiring presuppose the understanding and knowledge of Git. Moreover, it is free, so it can be used by individual developers, startups, as well as big organizations. There is only one situation in which I would consider using an alternative like Perforce and that would be in game development settings where binary resources are extremely large like textures, audio files, and 3D models and the Git LFS may fail to be efficient at scale. Until such niche cases, the obvious-choice is Git. Moreover, a self-hosted instance of GitLab is an enterprise-level solution that is the extension of Git with more powerful tools, but at the cost-efficient, open-source strategy.

In the case of contributing to Git itself, the entry cost is very high. Written in C and having a strong development culture it requires high quality patches and a strict development culture maintained by mailing lists. Nevertheless, with the consistent practice, I have gained a far greater explanation of Git than what I did prior to this audit. Giving back to Git is a heavy burden because the impact of any work on developers across the globe can be experienced.

So the first thing I would undertake is to use Git ecosystem it offers to build my tools, plugins, integrations, and automation solutions based on it and then finally would probably start to contribute to it.